

5 Konceptuální modelování (specializace WDOP)

1. Vysvětlíte rozdíl mezi konceptuální, logickou a fyzickou úrovní při návrhu dat. Uveďte pro každou úroveň co je jejím typickým výstupem a kdo ji typicky používá.
2. Navrhněte ER/UML model pro evidenci vládních usnesení: Existují Ministerstva (mají id a název) a Ministři (mají id a jméno). Každé ministerstvo má právě jednoho ministra. Existují Usnesení vlády (mají id, datum a název). Usnesení připravuje jedno nebo více ministerstev. U každé účasti ministerstva na přípravě usnesení evidujeme roli (garant, spoluředitel). Každé usnesení má právě jedno odpovědné ministerstvo. Navrhněte entity a jejich atributy, vztahy mezi entitami včetně kardinalit a určete, které vztahy mají vlastní atributy.
3. Transformujte navržený konceptuální model do relačního modelu. Uveďte tabulky, atributy, primární a cizí klíče a relevantní omezení. Navrhněte alespoň jednu alternativu pro vybranou část návrhu a vysvětlíte, kdy je daná alternativa vhodnější nebo méně vhodná.

Nástin řešení OTÁZKA 1.

1. Konceptuální úroveň: Popisuje sémantiku dat a strukturu domény nezávisle na technologii a datovém modelu (entity, vztahy, pravidla). Používá ji analytik, databázový návrhář a doménový expert. Výstupem je ER/UML model.
2. Logická úroveň: Popisuje strukturu dat v rámci zvoleného datového modelu (nejčastěji relačního): tabulky, klíče, vazby, integritní omezení. Používá ji databázový návrhář a vývojář. Výstupem je např. relační schéma.
3. Fyzická úroveň: Popisuje konkrétní implementaci v DBMS: uložení dat, indexy a optimalizaci výkonu. Používá ji DBA a performance specialista. Výstupem jsou fyzické DDL skripty a indexy.

OTÁZKA 2.

ENTITY:

MINISTERSTVO

ministerstvo_id (primární klíč)
název

MINISTR

ministr_id (primární klíč)
jméno

USNESENÍ

usneseni_id (primární klíč)
datum

název

VZTAHY:
ŘÍDÍ (MINISTR - MINISTERSTVO)
Typ vztahu: 1 : N
Kardinality: MINISTR -> MINISTERSTVO: (0..1) a MINISTERSTVO -> MINISTR: (1..1)
Význam: Každé ministerstvo má právě jednoho ministra, ministr může řídit nejvýše jedno ministerstvo.

PŘIPRAVUJE (MINISTERSTVO - USNESENÍ)
Typ vztahu: M : N
Kardinality: MINISTERSTVO -> USNESENÍ: (0..N) a USNESENÍ -> MINISTERSTVO: (1..N)
Atribut vztahu: role (garant, spoluřešitel)
Význam: Usnesení připravuje alespoň jedno ministerstvo a u každé účasti evidujeme roli.

ODPOVÍDÁ_ZA (MINISTERSTVO - USNESENÍ)
Typ vztahu: 1 : N
Kardinality: MINISTERSTVO -> USNESENÍ: (0..N) a USNESENÍ -> MINISTERSTVO: (1..1)
Význam: Každé usnesení má právě jedno odpovědné ministerstvo.

OTÁZKA 3.

RELAČNÍ MODEL (základní varianta)
Tabulky:
MINISTR(ministr_id PK, jmeno)
MINISTERSTVO(ministerstvo_id PK, nazev, ministr_id UNIQUE NOT NULL)
USNESENI(usneseni_id PK, datum, nazev, odpovedne_ministerstvo_id NOT NULL)
PRIPRAVA(ministerstvo_id, usneseni_id, role, PK(ministerstvo_id, usneseni_id))

Cizí klíče:
MINISTERSTVO.ministr_id -> MINISTR.ministr_id
USNESENI.odpovedne_ministerstvo_id -> MINISTERSTVO.ministerstvo_id
PRIPRAVA.ministerstvo_id -> MINISTERSTVO.ministerstvo_id
PRIPRAVA.usneseni_id -> USNESENI.usneseni_id

ALTERNATIVA: Např. vztah **ŘÍDÍ** jako samostatná tabulka. Výhody: Umožní evidovat střídání ministrů v čase (více

6 R stromy (specializace WDOP)

1. Definujte R strom a vysvětlete, k čemu se používá.
2. Vysvětlete princip MBR (Minimum Bounding Rectangle), včetně alespoň jedné výhody a jedné nevýhody.
3. Je dán přetečený listový uzel R-stromu s maximální kapacitou $M = 4$. Vytvořte konkrétní příklad takového uzlu a objektů v něm. Na tomto příkladu vysvětlete, jak tuto situaci řešit pomocí vhodného algoritmu pro rozdělení uzlu R stromu. Vysvětlete, jaký je cíl tohoto algoritmu, a uveďte jeho časovou složitost. (Hodnotu minimální zaplněnosti m zvolte dle vlastního uvážení, ale tuto volbu zdůvodněte.)

Nástin řešení OTÁZKA 1.

R-strom je výškově vyvážená hierarchická indexová struktura pro vícerozměrná prostorová data, navržená jako zobecnění B-stromu pro práci s geometrickými objekty. Ukládá objekty ve formě jejich minimálních ohraničujících obdélníků (MBR) a organizuje je do stromové struktury, kde každý vnitřní uzel obsahuje položky (MBR, ukazatel na podstrom) a každý listový uzel obsahuje položky (MBR, ukazatel na objekt). Všechny listové uzly jsou na stejné úrovni a každý uzel (kromě kořene) obsahuje počet položek v intervalu $\langle m, M \rangle$, což zajišťuje vyváženost stromu.

R-strom se používá k efektivnímu zpracování prostorových dotazů, zejména oblastních dotazů (range queries), dotazů na průnik objektů a dotazů typu nejbližší soused, typicky v GIS systémech, mapových službách, CAD aplikacích a databázových systémech s podporou prostorových dat.

OTÁZKA 2.

MBR (Minimum Bounding Rectangle) je nejmenší osově zarovnaný obdélník, který zcela obsahuje daný objekt nebo množinu objektů. V R-stromu slouží MBR jako aproximační obálka podstromu a umožňuje během vyhledávání rychlé prostorové filtrování: pokud se MBR uzlu neprotíná s dotazovanou oblastí, celý podstrom lze bezpečně vynechat.

Hlavní výhody: výrazné omezení počtu přístupů k datům, zrychlení prostorových dotazů, rychlé omezení počtu prohledávaných uzlů. Hlavní nevýhoda: Překryvy MBR a existence „mrtvé plochy“, což může vést k procházení více větví stromu.

OTÁZKA 3.

Pro konkrétní data můžeme zvolit například Guttmanův algoritmus rozdělení uzlu. Nejprve se vyberou tzv. seedy, tedy objekty, které jsou od sebe prostorově co nejvíce oddělené. Tyto objekty inicializují dva nové uzly. Následně se zbývající objekty postupně přiřazují k jednomu z nových uzlů tak, aby se minimalizovalo zvětšení MBR a překryv mezi uzly.

Volba parametru m (minimální zaplněnost uzlu) může ovlivnit výsledné rozdělení. V některých případech je nutné přiřadit objekt do skupiny s horším lokálním výsledkem (například s větším překryvem), aby byla splněna podmínka minimální zaplněnosti, i když by přiřazení do druhé skupiny vedlo ke lepšímu prostorovému rozdělení.

7 REST API pro streamovací aplikaci (specializace WDOP)

Uvažujte následující webové (REST) API inspirované službou Spotify. Všechny odpovědi jsou ve formátu JSON.

- GET /api/me/top/artists
Vrátí pole identifikátorů interpretů, např. ["a1", "a2", "a3"]
- GET /api/artists/{artistId}
Vrátí detail interpreta, např. { "id": "a123", "name": "Radiohead", "popularity": 87, "genres": ["alternative", "rock"] }
- GET /api/artists/{artistId}/related-artists
Vrátí pole interpretů (objekty), např. [{ "id": "a9", "name": "Thom Yorke"}, { "id": "a8", "name": "Portishead"}]
- GET /api/artists/{artistId}/top-tracks
Vrátí pole skladeb (objekty) pro daného interpreta, např. [{ "id": "t1", "name": "Karma Police"}, { "id": "t2", "name": "No Surprises"}]

1. Vysvětlete, co je to REST API a jaký je význam HTTP metod GET a POST při návrhu webových aplikací.
2. Napište v JavaScriptu (kód poběží ve webovém prohlížeči) funkci, která:
 - (a) získá seznam identifikátorů uživatelských top interpretů,
 - (b) pro každý identifikátor stáhne jméno interpreta,
 - (c) vypíše všechna tato jména do HTML elementu ul s id top-artists.

Drobné syntaktické chyby nemají vliv na hodnocení. Pokud si nejste jistí názvem metody, popište její funkci.

3. Napište samostatnou JavaScriptovou funkci, která:
 - (a) dostane jako argument artistId,
 - (b) získá seznam souvisejících interpretů,
 - (c) vypíše jejich jména do HTML elementu ul s id related-artists.

Drobné syntaktické chyby nemají vliv na hodnocení. Pokud si nejste jistí názvem metody, popište její funkci.

4. Stručně vysvětlete, jak by bylo možné na základě výše uvedeného API vytvořit jednoduché doporučení dalších skladeb aktuálnímu uživateli. (Popis postačí na úrovni algoritmu, není nutná implementace.)

Nástin řešení

1. REST API je rozhraní založené na práci s prostředky identifikovanými URL. Metoda GET slouží ke čtení dat, metoda POST k vytvoření nebo změně dat na serveru.

2. Základní postup:

```
async function fetchJson(url) {
  return await (await fetch(url)).json();
}

async function showTopArtists() {
  const ids = await fetchJson("./api/me/top/artists");
  const ul = document.getElementById("top-artists");

  for (const id of ids) {
    const artist = await fetchJson("./api/artists/" + id);
    const li = document.createElement("li");
    li.innerText = artist.name;
    ul.appendChild(li);
  }
}
```

3. Základní postup:

```
async function showRelatedArtists(artistId) {
  const related = await fetchJson(
    "./api/artists/" + artistId + "/related-artists"
  );
  const ul = document.getElementById("related-artists");

  for (const a of related) {
    const li = document.createElement("li");
    li.innerText = a.name;
    ul.appendChild(li);
  }
}
```

4. Doporučení lze vytvořit např. tak, že pro každého uživatele oblíbeného interpreta získáme jeho související interprety a ty agregujeme (např. podle četnosti výskytu nebo popularity). Ze seznamu odstraníme interprety kteří se již vyskytují v odpovědi na `/api/me/top/artists`, čímž získáme seznam interpretů, kteří nejsou mezi oblíbenými, ale jsou podobní těm, které uživatel poslouchá. Pro ně pak získáme top skladby a nad výsledným seznamem např. provedeme random sampling vážený důležitostí interpreta.

8 Doporučovací systémy (specializace WDOP)

- Vysvětlete princip fungování *user-based k-nearest neighbors* (UB-KNN) algoritmu pro kolaborativní filtrování. Uveďte alespoň 2 **nevýhody** tohoto přístupu.
- Uvažujte následující matici explicitních hodnocení (vyšší číslo znamená lepší hodnocení, symbol „-“ znamená, že uživatel položku nehodnotil):

uživatel \ položka	I1	I2	I3	I4	I5	I6
U_0 (cílový)	5	3	-	-	-	-
U_1	5	3	5	1	-	-
U_2	4	-	4	5	-	-
U_3	1	1	-	5	5	-
U_4	5	4	-	-	5	1

Předpokládejte doporučování pomocí *user-based KNN* s počtem sousedů $k = 2$ pro cílového uživatele U_0 .

- (a) Zvolte metriku podobnosti uživatelů  a stručně popište, jak se počítá. Určete nejbližší sousedy pro uživatele U_0 .

¹V tomto kroku existuje řada akceptovatelných variant. Vyberte libovolnou metriku, která je běžně používaná v referenčních implementacích – případně takovou, jejíž vhodnost dokážete dostatečně vysvětlit.